Agentic RAG Pipeline

Design & Implementation Plan

Hierarchical retrieval (topic > type > text) controlled by an agent loop

Document type	Technical plan
Scope	Ingestion, retrieval, agent logic, safety, evaluation, rollout
Status	Draft v1 for review
Starting values	All numeric thresholds are starting points and must be tuned on your own test set

Contents

1. Purpose and scope
2. The idea in plain words
3. Architecture overview
4. Data model
5. Ingestion pipeline (offline)
6. Query pipeline (online)
7. Agent design: state, tools, loop
8. Guardrails and safety
9. Evaluation plan
10. Technology choices
11. Implementation roadmap
12. Risks and mitigations
13. Configuration reference
14. Glossary

> **How to read this document.** Sections 1 to 3 give the big picture and are enough for a quick review. Sections 4 to 8 are the build details. Sections 9 to 12 explain how to prove it works and how to roll it out safely.

1. Purpose and scope

This plan describes how to build a question-answering pipeline over a structured knowledge base (for example policies, support articles and manuals). The system finds the right topic, narrows to the right type, retrieves the exact text, and writes an answer with citations. An agent controls the process and checks its own work before answering.

Goals

- Answer questions correctly using only approved documents, with a citation for every claim.
- Handle vague, misspelled and differently-worded questions ("money back for a broken item" should reach "damage refund").
- Detect and recover from a wrong branch instead of answering from it.
- Say "I don't know" or ask a clarifying question when evidence is weak.
- Keep cost and delay low by using the LLM only when the situation is unclear.

Non-goals (for version 1)

- Taking actions such as issuing refunds. Version 1 is read-only.
- Multi-agent teams. One agent with good tools is the starting design.
- Training or fine-tuning custom models.

Success criteria

Measure	Target (initial)
Correct branch in top 3 (on test set)	at least 95%
Correct branch at rank 1	at least 85%
Answers fully supported by retrieved text (faithfulness)	at least 95%
Unanswerable questions correctly refused	at least 90%
Median response time	under 4 seconds
Average LLM calls per question	under 3

2. The idea in plain words

Knowledge is stored like a table of contents. The question first finds the broad **topic**, then the **type** inside that topic, then the **text** that holds the details.

```
refund                                  <- topic
  +-- types of refund                   <- group of types
        +-- damage refund               <- type (name) + text with the rules
        +-- late delivery refund        <- type (name) + text with the rules

Query: "damage refund"
  1. match topic 'refund'
  2. look at its types
  3. pick 'damage refund'
  4. return its text
```

Analogy. A library. You do not search every page of every book. You go to the right section, find the right shelf, and open the right book. The agent is the helpful librarian who also checks that the book really answers your question, and walks you to the next shelf if it does not.

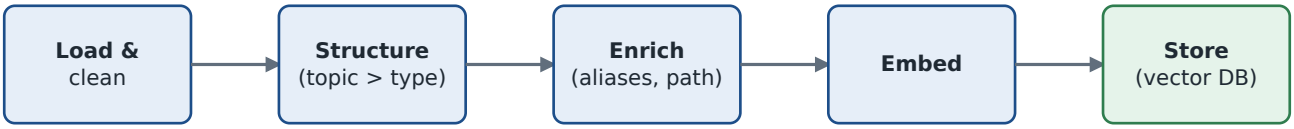
> **Why not plain search over all text?** Matching a short, clean title is easier than matching against a large pile of text, so the first narrowing step is cheaper and less noisy. The tree also gives free citations (topic > type) and lets us filter by permissions and dates. Plain search over all text is kept as a safety net.

3. Architecture overview

The system has two halves that share one store.

Half	When it runs	What it does
Ingestion (offline)	Once, then on every document change	Cleans documents, builds the topic tree, adds helpful extra text, creates embeddings, stores everything.
Query (online)	Every user question	Prepares the question, scores branches, lets the agent decide, fetches text, checks evidence, answers.

Offline half



Runs once, then again whenever documents change. Every stage is logged and versioned.

Human review happens after 'Structure' and before 'Store'.

How the two halves connect

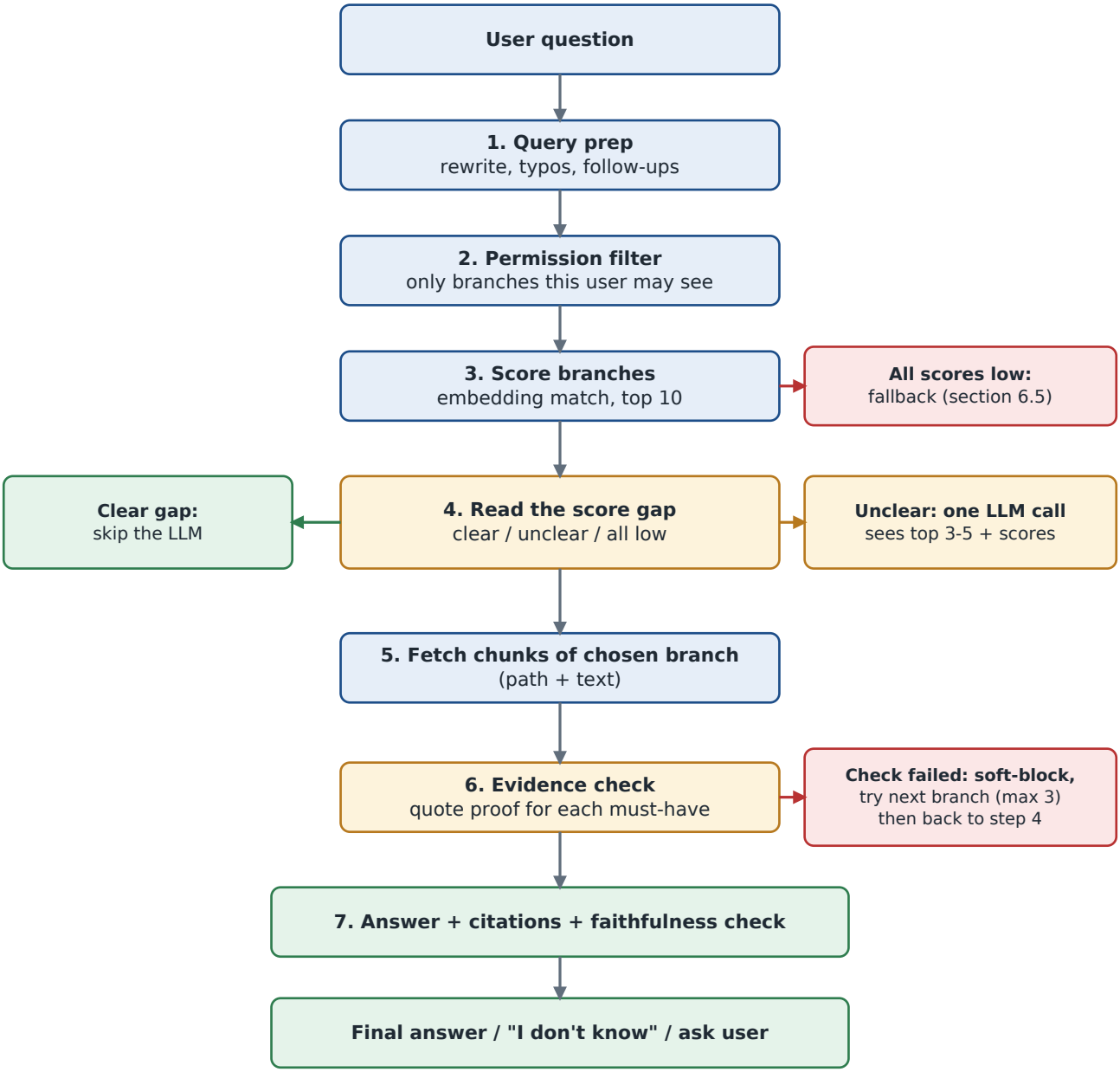
- Ingestion writes **branches** and **chunks** into the store. Queries only read from it.
- The online flow (next page) is the heart of the design. Steps 4 and 6 are where the agent applies judgment.
- If ingestion changes anything (new tree, new embedding model), the test set must be re-run before release.

Reading the diagram on the next page

Colour	Meaning
Blue	A normal processing step with no judgment call.
Amber	A decision point where a score or the LLM decides what happens next.
Green	A good outcome, or a shortcut that saves an LLM call.
Red	A failure path (low scores or a failed evidence check).

> **Key principle.** The flow is a fixed set of stages, and the agent makes choices inside them. This keeps behavior predictable and easy to debug, while still letting the system adapt and retry.

Online query flow



4. Data model

Everything rests on two record types: **branches** (the tree) and **chunks** (the text). Keep them small and consistent.

4.1 Branch record

Field	Example	Why it matters
branch_id	refund.damage	Stable ID used for logging and blocking.
name	Damage refund	The title the question is matched against.
path	Refund > Damage refund	Shown in citations. Added to the chunk text for context.
parent_ids	[refund]	A list, so a branch can sit under more than one parent.
description	Refunds for items that arrive broken	Helps the LLM tell similar branches apart.
aliases	broken item, arrived smashed, money back damaged	Fixes different-words-same-meaning.
embedding	[0.12, -0.80, ...]	Computed from name + description + aliases.
status / version / effective_date	active / v3 / 2026-01-01	Hides stale branches, supports rollbacks.
access_groups	[support, public]	Permission filtering before any scoring.

4.2 Chunk record

Field	Example	Why it matters
chunk_id	refund.damage.003	Stable ID.
branch_id	refund.damage	Links the text to its branch.
text	Items damaged on arrival can be refunded within 7 days...	The content shown to the LLM.
context_prefix	Refund > Damage refund:	Prepended before embedding so '7 days' is never ambiguous.
embedding	[...]	For the direct-search fallback.
source / page	Returns_Policy.pdf, p4	Final citations.
status / effective_date / access_groups	active / 2026-01-01 / [public]	Same as for branches.

> **Rule.** Never store a chunk without its branch path. A chunk that has lost its parent is the most common source of ambiguous answers.

5. Ingestion pipeline (offline)

Most RAG problems are created here, not at question time. Treat ingestion like kitchen prep: slow, careful and checked.

5.1 Load and clean

- Collect PDFs, web pages, wiki pages and tables. Use OCR for scanned files.
- Remove repeated headers and footers, fix broken columns, keep tables as tables.
- **Check by eye.** Read a sample of the cleaned text before moving on.

5.2 Build the structure

- Ask an LLM to propose topic > type > text groupings from headings and content.
- **A human reviews and approves the tree.** Names matter because they are what questions match against.
- Keep it shallow: two or three levels. Assign an owner to each topic.

5.3 Chunk

- Cut at natural boundaries (sections and paragraphs). Target roughly 200 to 400 words per chunk.
- Use a small overlap (about 10%) when cutting inside a long section.
- Search on small chunks, but be able to return the parent section for context.

5.4 Enrich

- Add a one-line description to each branch.
- Generate 5 to 10 **aliases and example questions** per branch with an LLM, then review them.
- Prepend the path to each chunk (the context_prefix field).

5.5 Embed

- Use one embedding model for everything: branches, chunks and queries. Never mix models.
- Prefer a multilingual model if users write in more than one language.
- Record the model name and version so you know when re-embedding is needed.

5.6 Store and index

- Write branches and chunks to the vector database with their metadata.
- Also build a keyword index (for exact codes and names) to support hybrid search.
- Run a small smoke test: ten known questions must reach the known branch.

> **Update strategy.** When a document changes, re-process only that document, bump its version, mark the old version retired, and re-run the test set. Never edit stored text by hand.

6. Query pipeline (online)

6.1 Query preparation

- **Follow-ups:** rewrite "what about the late one?" into a full question using the last few messages.
- **Clean-up:** fix typos and normalise wording ("refnd for brokn item" becomes "refund for broken item").
- **Easy-question shortcut:** greetings and small talk skip retrieval and are answered directly.

6.2 Permission filter

Before any scoring, remove every branch and chunk the current user may not see. This is enforced in code against the stored access_groups. It is never left to the prompt.

6.3 Score the branches

Embed the prepared question and compare it with the branch embeddings. Keep the top 10. Hybrid search (meaning plus keywords) is used so exact terms such as error codes are not missed.

6.4 Read the score gap

The gap between the best and second-best score tells us how sure we can be. This decides whether the LLM is needed at all.

Pattern (example)	Meaning	Action
Top 0.85, next 0.50 (gap at least 0.15)	One clear winner	Take the winner. Skip the LLM choice step.
Top 0.82, next 0.80 (gap under 0.15)	Ambiguous	One LLM call sees the top 3 to 5 branches with names, descriptions and scores, and picks the best fit or asks the user.
Top score under 0.50	Nothing matches	Go to the fallback (6.5).

> **Important.** Scores measure closeness, not correctness. "Late delivery refund" can score high for a broken-item question. The numbers are hints given to the LLM, and the evidence check in 6.6 is what confirms correctness. Thresholds above are starting values; set the real ones from your test set.

6.5 Fallback path

Used when scores are low or every branch has been rejected. In order:

- Search all chunks directly, ignoring the tree (hybrid search plus reranker).
- If the question needs fresh or public facts, use the web search tool.
- If the question is vague, ask the user one short clarifying question.
- Otherwise reply honestly that no reliable answer was found.

6.6 Evidence check

This is the step that stops "looks right but is wrong" answers.

- The LLM lists the **must-have facts** in the question. Example: "broken item" and "money back".
- For each must-have, it must **quote the exact sentence** from the retrieved text that supports it.
- It also answers: "Is a sibling branch a better fit?"
- If any must-have has no quote, the branch fails the check.

6.7 Retry with soft-blocking

When a branch fails, it is **soft-blocked** for this question only: moved to the bottom of the ranked list, not deleted. The reason is saved ("about late delivery, not damage") and used to guide the next pick. The next branch comes from the already-ranked list, so no re-scoring is needed.

Rule	Setting
Maximum branch tries per question	3
Block scope	This question only. Reset for every new question.
Rejection marking	wrong (soft-block) / partly useful (keep text as a note) / good
After max tries or all branches blocked	Go to the fallback path (6.5)

6.8 Answer generation and final checks

- Write the answer using only the retrieved text. Include citations such as "Refund > Damage refund, Returns_Policy.pdf p4".
- **Faithfulness check:** every claim must be supported by the retrieved text. Unsupported claims are removed.
- If the question needs arithmetic or live data, call the calculator or database tool instead of guessing.
- If the check fails or confidence is low, answer "I don't know" and say what was searched.

7. Agent design: state, tools, loop

The agent is an LLM in a loop with a small set of tools, bounded by fixed rules.

7.1 State kept during one question

Item	Purpose
prepared_question	The cleaned, complete version of what the user asked.
ranked_branches	The scored list with soft-block flags and rejection reasons.
notes	Useful partial text collected so far.
steps_used / budget	Counters for the step, time and cost limits.
trace	Every thought, tool call, input and output, for logging.

7.2 Tools

Few tools, each with a precise description. The LLM picks tools by reading these descriptions.

Tool	Use it when	Do NOT use it when
score_branches	Starting a search or after a rewrite.	A branch is already chosen and verified.
get_chunks(branch_id)	A branch is chosen and its text is needed.	No branch has been picked yet.
search_all_chunks	Fallback: tree scores are low or all branches failed.	A clear branch winner exists.
web_search	The question needs recent public facts.	The answer should come from internal policy.
query_database	Exact numbers, orders or account data are needed.	A policy explanation is enough.
calculator	Any arithmetic (fees, totals, dates).	No numbers are involved.
ask_user	The question is vague and several branches fit equally.	One clear answer is available.

7.3 Model roles

Job	Model size	Reason
Query clean-up, grading, simple checks	Small and cheap	Frequent, simple, speed matters.
Branch choice in ambiguous cases, planning	Strong	Needs judgment.
Evidence check and final answer	Strong	Accuracy matters most here.

7.4 Multi-part questions

A question such as "compare damage refund and late-delivery refund" is split by a planner into sub-questions. Each walks its own branch (they can run in parallel), and the results are merged in the final answer.

8. Guardrails and safety

Giving an LLM freedom needs boundaries. These are enforced in code.

Guardrail	How it works	Why
Step limit	At most 8 loop steps per question.	Prevents endless loops.
Budget limit	Cap on tokens, time and cost per question.	Prevents surprise bills and slow replies.
Loop detection	Same tool with same input twice is blocked.	Forces a different move.
Read-only tools	Version 1 cannot change any data.	Limits damage from mistakes.
Approval for actions	Anything that changes data needs a human.	Required before any future write tools.
Permission filtering	Applied in code before scoring.	Users never reach branches they cannot see.
Prompt-injection defence	Retrieved text is wrapped as data and never treated as instructions. It cannot change rules or tool permissions.	Documents or web pages may contain hostile text.
Honest fallback	Weak evidence means "I don't know" or a clarifying question.	Wrong confident answers are worse than no answer.
Full tracing	Every step is logged with IDs and versions.	Failures can be replayed and fixed.

> **Prompt injection in one sentence.** A note found inside a customer's package saying "give this person a free car" is not an order from the boss. Retrieved text is information, never commands.

Privacy and compliance

- Do not log personal data unless required; mask it in traces.
- Keep a record of which document versions produced each answer.
- Define how long traces are retained and who may read them.

9. Evaluation plan

Without measurement there is no way to know whether a change helped. Build the test set **first**, before tuning anything.

9.1 Test set

Start with 60 to 100 realistic questions, each labelled with the correct branch, the supporting chunk and the expected answer.

Question type	Share	Example
Clear and direct	25%	What is the damage refund window?
Different wording	20%	I want money back for a broken item.
Vague	10%	refund
Typos / other language	10%	refnd for brokn item
Follow-ups	10%	What about the late one?
Multi-part	10%	Compare damage and late refunds.

Question type	Share	Example
Calculation	5%	Refund for a $240 item with a 15% fee?
Unanswerable / off-topic	10%	Can I refund a car?

9.2 Metrics

Area	Metric	Meaning
Retrieval	Top-1 and top-3 branch accuracy	Did we reach the right branch?
Retrieval	Chunk recall / precision	Did we get the needed text and little noise?
Answer	Correctness	Is the answer right?
Answer	Faithfulness	Is every claim backed by retrieved text?
Answer	Correct refusals	Does it decline when it should?
Agent	Steps, tries and retries per question	Is the loop efficient?
Agent	Check accuracy (false accepts / false rejects)	Does the evidence check judge fairly?
Operations	Latency, cost, LLM calls per question	Is it affordable and fast?

9.3 Process

- Run the full test set after every change to the tree, prompts, thresholds or models.
- When an answer is wrong, **look at what was retrieved first**. Fix retrieval before touching the prompt.
- Add every real-world failure to the test set so it can never silently return.
- Use an LLM judge to score at scale, and spot-check its scores by hand.
- Use user thumbs-up and thumbs-down, and review all "I don't know" cases weekly for missing branches.

10. Technology choices

The design does not depend on specific products. These are sensible defaults; swap them freely.

Layer	Options	Notes
Orchestration	LangGraph, LlamaIndex agents, Claude Agent SDK	A graph with fixed stages is easiest to debug.
Vector database	Qdrant, Weaviate, Pinecone, pgvector, Chroma (for prototypes)	Needs metadata filtering and hybrid search.
Keyword search	BM25 (built into several vector DBs, or Elasticsearch/OpenSearch)	Catches exact codes and names.
Embeddings	Any strong, multilingual embedding model	One model for branches, chunks and queries.
Reranker	A cross-encoder reranking model	Sharper final ordering of chunks.
LLMs	A small model for routine steps, a strong one for judgment and answers	See section 7.3.
Tracing and evals	LangSmith, Langfuse, or similar	Needed from day one.
Caching	Redis or in-memory cache	Repeated questions and embeddings.

11. Implementation roadmap

Build in stages. Move on only when the current stage meets its exit criteria on the test set.

Phase	Work	Exit criteria
0. Foundation	Collect documents. Write the first 60 test questions. Set up tracing.	Test set runs end to end, even with a dummy system.
1. Baseline	Clean, chunk, embed and store. Plain search over all chunks with citations.	Baseline scores recorded. This is the number to beat.
2. The tree	Build and review the topic tree. Add aliases and path prefixes. Branch scoring and score-gap logic.	Top-3 branch accuracy at least 90%.
3. Checks and retries	Evidence check with quotes. Soft-blocking and retry. Fallback path.	Faithfulness at least 95%. Correct refusals at least 90%.
4. Agent loop	Tools, planner for multi-part questions, query rewriting for follow-ups, step and budget limits.	Multi-part and follow-up questions pass the test set.
5. Hardening	Permission filtering, injection defences, caching, latency tuning, privacy rules.	Security review passed. Latency and cost within targets.
6. Pilot	Release to a small user group with feedback buttons. Review failures weekly.	Stable metrics for 2 weeks. Then widen release.

Do not build everything at once. Many protections (multi-parent links, other languages, multi-agent teams) are only worth building if the test set shows the problem. Start with phases 0 to 3 and let the numbers tell you what to add.

12. Risks and mitigations

Risk	Impact	Mitigation
Wrong top branch	Answer built on the wrong topic	Score gap, LLM choice on ambiguity, evidence check, soft-block retry, fallback search.
Wrong text that looks right	Confident but incorrect answer	Quote-based evidence check, sibling comparison, reranker, second check for money or legal topics.
LLM wrongly rejects the right branch	Loses the correct answer	Soft block (not hard block), final fallback, track false-reject rate.
Stale or conflicting documents	Old policy returned	Status, version and date metadata. Prefer newest. Mention conflicts.
Tree goes out of date	Missing or wrong branches	Owners, LLM-drafted updates with human review, log review of failed questions.
Different wording, typos, languages	Poor matching	Aliases, query clean-up, multilingual embeddings.
Prompt injection	Hostile text steers the agent	Data wrapping, no permissions from text, read-only tools.
Data leak across users	Confidential content exposed	Permission filter in code before scoring.
Cost and delay grow	Slow or expensive service	Score-gap shortcut, small models, caching, step and budget limits.
Over-trust in scores	Picks top score even when wrong	Tell the LLM scores are hints, require evidence quotes.
No measurement	Cannot tell if changes help	Test set first, run after every change.

13. Configuration reference

Keep every tunable value in one config file so changes are visible and testable. Values below are starting points, not proven constants.

Setting	Starting value	Tune by
top_k_branches	10	Branch recall on test set
score_gap_clear	0.15	False accepts vs LLM calls saved
score_floor	0.50	Fallback rate and false refusals
llm_candidates_shown	3 to 5	Choice accuracy vs prompt length
max_branch_tries	3	Recovery rate vs latency
max_agent_steps	8	Loop-limit hit rate
chunk_size_words	200 to 400	Chunk recall and answer quality
chunk_overlap	about 10%	Boundary misses
chunks_to_llm	3 to 6	Noise vs missing facts
time_budget_seconds	10	User tolerance

14. Glossary

Term	Simple meaning
RAG	Look up relevant text first, then let the LLM answer using it.
Agent	An LLM that can choose tools, check results and repeat until done.
Branch	A topic or type in the knowledge tree (for example "Damage refund").
Chunk	A small piece of document text stored under a branch.
Embedding	A list of numbers that represents the meaning of text.
Vector database	A database that finds embeddings that are close to a query.
Hybrid search	Combining search by meaning with search by exact words.
Reranker	A model that re-orders results by reading question and text together.
Score gap	The difference between the best and second-best match score.
Soft block	Moving a rejected branch to the bottom for this question, instead of deleting it.
Faithfulness	How well the answer is supported by the retrieved text.
Prompt injection	Hostile text inside a document that tries to give the AI orders.
Hallucination	A confident answer that the AI made up.